CENTRO UNIVERSITÁRIO FEI

Avenida Humberto de Alencar Castelo Branco, 3972, CEP: 09850-901 São Bernardo do Campo
Telefone: (011) 4353-2900 Fax (011) 4109-5994

Curso de Ciências da Computação

Empréstimo de Livros

Construção de Serviços

Autores: Lucas Kerr Marcela Nalesso Mariane Carvalho	Data de emissão: 29/04/2026
Revisor:	Data de revisão

Índice

1 OBJETIVO DO DOCUMENTO **3**

2 ESTILO DE COORDENAÇÃO DOS SERVIÇOS **3**

3 ESTRATÉGIA DE TESTE **4**

Tipo de Teste: Teste Unitário 4
Tipo de Teste: Teste de Integração 4

4 EVIDÊNCIAS DOS TESTES REALIZADOS (PRINTS) **4**

1 Objetivo do Documento

Este documento tem como objetivo descrever a implementação de dois serviços em uma arquitetura orientada a serviços, incluindo a definição do estilo de comunicação entre serviços, justificativa arquitetural e definição de estratégias de teste.

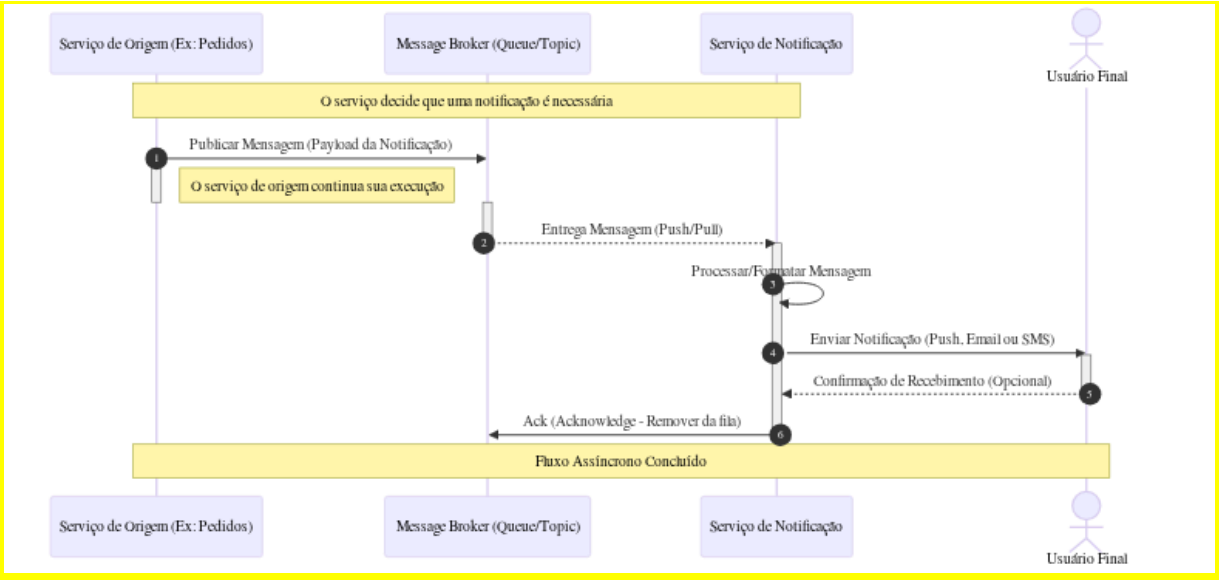
2 Estilo de Coordenação dos Serviços

Escolhido: Orquestração

- **Justificativa da escolha**:
  A orquestração foi escolhida porque permite controle centralizado do fluxo do sistema, concentrando a coordenação das operações em um único componente. Isso reduz a complexidade e facilita a manutenção.
  Além disso, torna o sistema mais fácil de monitorar e auditar, já que todas as execuções passam pelo orquestrador, permitindo rastrear ações, identificar falhas e registrar logs de forma clara e organizada.

- **Como ocorre a comunicação entre os serviços:**
  Utilizaremos um modelo de comunicação assíncrona, estilo Pub-Sub, em que teremos um serviço centralizado que envia a mensagem e os envolvidos no sistema recebem.

- **Papel de cada serviço no fluxo:**
  - **enviarNotificacao():** Envia mensagens de notificação aos usuários.
  - **atualizarQuantidadeLivro():** Atualiza o valor de estoque de livros caso haja empréstimo ou renovação.
- Diagrama ou fluxo representando a interação entre os serviços:



3 Estratégia de Teste

Tipo de Teste: Teste Unitário (Serviço: atualizarQuantidadeLivro)

Objetivo do Teste:	Validar a lógica de atualização de estoque de livros de forma isolada, garantindo que a quantidade seja corretamente incrementada ou decrementada em operações de empréstimo e renovação.
Técnica:	• Testes unitários com isolamento total das dependências • Uso de mocks para o repositório de livros • Execução direta do método `atualizarQuantidadeLivro()` • Uso de dados simulados (quantidade inicial, operação realizada) • Sem acesso ao banco de dados real
Critério de Finalização	• Todas as regras de atualização testadas individualmente • Casos positivos (atualização correta) e negativos (estoque insuficiente) cobertos • Testes executados sem falhas

Tipo de Teste: Teste de Integração (Serviços: atualizarQuantidadeLivro + enviarNotificacao)

Objetivo do Teste:	Verificar se, ao atualizar a quantidade de livros, o sistema aciona corretamente o envio de notificação ao usuário.
Técnica:	● Execução de chamadas reais ou simuladas entre os serviços ● Validação do fluxo: atualização de estoque → envio de notificação ● Verificação do conteúdo da mensagem enviada ● Simulação de cenários (sucesso e falha na notificação)
Critério de Finalização	● Atualização de estoque realizada corretamente ● Notificação enviada após a operação ● Comunicação entre serviços sem erros ● Todos os cenários executados com sucesso ● Falhas tratadas adequadamente

4 Evidências dos testes realizados (prints)

```python
class LivroRepository:
    def __init__(self):
        # Simulando banco de dados em memória
        self.livros = {
            1: {"titulo": "Engenharia de Software", "quantidade": 5},
            2: {"titulo": "Redes de Computadores", "quantidade": 3},
        }

    def buscar_livro(self, livro_id):
        return self.livros.get(livro_id)

    def atualizar_quantidade(self, livro_id, nova_quantidade):
        if livro_id in self.livros:
            self.livros[livro_id]["quantidade"] = nova_quantidade
```

```python
class NotificacaoService:
    def enviarNotificacao(self, usuario, mensagem):
        # Simulação de envio (pode ser e-mail, SMS, etc)
        print(f"[NOTIFICAÇÃO] Para: {usuario} | Mensagem: {mensagem}")
```

```python
class LivroService:
    def __init__(self, repository, notificacao_service):
        self.repository = repository
        self.notificacao_service = notificacao_service

    def atualizarQuantidadeLivro(self, livro_id, operacao, usuario):
        livro = self.repository.buscar_livro(livro_id)

        if not livro:
            raise Exception("Livro não encontrado")

        quantidade_atual = livro["quantidade"]

        if operacao == "emprestimo":
            if quantidade_atual <= 0:
                raise Exception("Estoque insuficiente")
            nova_quantidade = quantidade_atual - 1

        elif operacao == "devolucao":
            nova_quantidade = quantidade_atual + 1

        else:
            raise Exception("Operação inválida")

        # Atualiza estoque
        self.repository.atualizar_quantidade(livro_id, nova_quantidade)

        # Envia notificação
        mensagem = f"O livro '{livro['titulo']}' foi atualizado. Nova quantidade: {nova_quantidade}"
        self.notificacao_service.enviarNotificacao(u[illegible]io, mensagem)

        return nova_quantidade
```

Teste de Execução das Funções:

```python
if __name__ == "__main__":
    repo = LivroRepository()
    notificacao_service = NotificacaoService()
    livro_service = LivroService(repo, notificacao_service)

    # Empréstimo
    livro_service.atualizarQuantidadeLivro(1, "emprestimo", "usuario@universidade.edu")

    # Devolução
    livro_service.atualizarQuantidadeLivro(1, "devolucao", "usuario@universidade.edu")
```